Hack Todo List に必要なデータベース

結論

- 必須DBは **Supabase PostgreSQL**（1つ）で十分。
- 認証は Supabase Auth、業務データは PostgreSQL テーブルで管理する。

必須テーブル一覧

#	テーブル名	目的	役割	主なカラム（抜粋）
1	`profiles`	ユーザー追加情報・設定管理	`auth.users` を補完するプロフィール基盤	`id (uuid, PK, auth.users.id)` / `display_name (text)` / `timezone (text)` / `created_at (timestamptz)`
2	`todos`	Todo管理	タスク保存、進捗・優先度管理の中核データ	`id (uuid, PK)` / `user_id (uuid, FK)` / `title (text)` / `description (text)` / `due_date (date)` / `status (todo/doing/done)` / `priority (smallint)` / `created_at` / `updated_at`
3	`calendar_events`	カレンダー機能・日付管理	予定を時系列で可視化するイベントデータ	`id (uuid, PK)` / `user_id (uuid, FK)` / `title (text)` / `start_at` / `end_at` / `all_day (boolean)` / `created_at`
4	`tab_sessions`	タブ滞在ログ保存	URLごとの滞在秒数を記録する行動ログ（一次データ）	`id (bigserial, PK)` / `user_id (uuid, FK)` / `domain` / `url` / `tab_title` / `started_at` / `ended_at` / `duration_sec` / `is_waste` / `created_at`
5	`alert_rules`	警告条件管理	ドメインごとの警告しきい値を管理	`id (uuid, PK)` / `user_id (uuid, FK)` / `target_domain` / `threshold_sec` / `enabled` / `created_at`
6	`btc_price_history`	BTC価格履歴管理	金額換算の再計算・検証用の参照データ	`id (bigserial, PK)` / `source` / `price_jpy (numeric)` / `fetched_at`
7	`waste_cost_snapshots`	浪費時間の金額換算結果	グラフ表示向けの集計済みKPIデータ	`id (bigserial, PK)` / `user_id (uuid, FK)` / `period_start` / `period_end` / `wasted_seconds` /

#	テーブル名	目的	役割	主なカラム（抜粋）
				`btc_price_jpy` / `lost_amount_jpy` / `created_at`

インデックス（最低限）

- tab_sessions(user_id, started_at)
- todos(user_id, due_date)
- calendar_events(user_id, start_at)
- waste_cost_snapshots(user_id, period_start)

セキュリティ

- Supabase RLS を有効化する。
- すべてのユーザーデータ系テーブルは「自分の user_id のみ SELECT/INSERT/UPDATE/DELETE 可」にする。

補足（任意）

- 将来データ量が増えたら tab_sessions を月単位でパーティション化。
- 高速通知やレート制御が必要なら Redis を追加検討。

カラムの意味（わかりやすい説明）

この設計では、認証・Todo・カレンダー・タブ監視・警告・BTC換算をそれぞれ別テーブルで管理します。

- `profiles` はユーザーの基本設定
- `todos` はタスク本体
- `calendar_events` は予定・日付管理
- `tab_sessions` はブラウザ滞在ログ
- `alert_rules` は警告条件
- `btc_price_history` はBTC価格の履歴
- `waste_cost_snapshots` は浪費時間の金額換算結果

1. profiles のカラム説明（表形式）

カラム	型/制約	意味
`id`	uuid, PK (`auth.users.id` と同一)	ユーザー本人を識別するID
`display_name`	text	画面に表示するユーザー名
`timezone`	text	日時表示の基準となるタイムゾーン
`created_at`	timestamptz	ユーザー登録日時

2. todos のカラム説明（表形式）

カラム	型/制約	意味
`id`	uuid, PK	Todoの一意ID

カラム	型/制約	意味
`user_id`	uuid, FK -> `profiles.id`	どのユーザーのTodoか
`title`	text	タスク名
`description`	text	タスクの詳細説明
`due_date`	date	締切日
`status`	text (`todo` / `doing` / `done`)	進捗状態
`priority`	smallint	優先度
`created_at`	timestamptz	Todo作成日時
`updated_at`	timestamptz	Todo更新日時

3. calendar_events のカラム説明（表形式）

カラム	型/制約	意味
`id`	uuid, PK	予定の一意ID
`user_id`	uuid, FK -> `profiles.id`	どのユーザーの予定か
`title`	text	予定名
`start_at`	timestamptz	開始日時
`end_at`	timestamptz	終了日時
`all_day`	boolean	終日予定かどうか
`created_at`	timestamptz	予定の登録日時

4. tab_sessions のカラム説明（表形式）

カラム	型/制約	意味
`id`	bigserial, PK	タブ滞在ログの一意ID
`user_id`	uuid, FK -> `profiles.id`	どのユーザーの行動ログか
`domain`	text	`youtube.com` などのドメイン
`url`	text	実際に開いていたページURL
`tab_title`	text	タブに表示されるページタイトル
`started_at`	timestamptz	滞在開始時刻
`ended_at`	timestamptz	滞在終了時刻
`duration_sec`	integer	滞在秒数
`is_waste`	boolean	浪費時間として扱うか
`created_at`	timestamptz	ログ保存日時

5. alert_rules のカラム説明（表形式）

カラム	型/制約	意味
`id`	uuid, PK	警告ルールの一意ID
`user_id`	uuid, FK -> `profiles.id`	どのユーザー向け設定か
`target_domain`	text	監視対象ドメイン
`threshold_sec`	integer	警告を出す秒数のしきい値
`enabled`	boolean	ルールが有効か無効か
`created_at`	timestamptz	ルール登録日時

6. btc_price_history のカラム説明（表形式）

カラム	型/制約	意味
`id`	bigserial, PK	BTC価格履歴の一意ID
`source`	text	取得元のAPIやサービス名
`price_jpy`	numeric	その時点のBTC価格（日本円）
`fetched_at`	timestamptz	価格を取得した時刻

7. waste_cost_snapshots のカラム説明（表形式）

カラム	型/制約	意味
`id`	bigserial, PK	集計結果の一意ID
`user_id`	uuid, FK -> `profiles.id`	どのユーザーの集計か
`period_start`	timestamptz	集計開始時刻
`period_end`	timestamptz	集計終了時刻
`wasted_seconds`	integer	浪費時間の合計秒数
`btc_price_jpy`	numeric	換算に使ったBTC価格
`lost_amount_jpy`	numeric	失った金額の計算結果
`created_at`	timestamptz	集計作成日時

この設計で実現できること

- ログイン・登録: `profiles`
- Todo管理と締切表示: `todos`
- カレンダー表示: `calendar_events`
- タブごとの浪費時間グラフ: `tab_sessions`
- 一定時間超過の警告: `alert_rules`
- BTCや円換算の損失グラフ: `btc_price_history` と `waste_cost_snapshots`